1. Write the Python program to solve the 8-Puzzle problem (CO1).

```
from collections import deque
start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)
queue = deque([(start, [])])
visited = {start}
while queue:
    state, path = queue.popleft()

    if state == goal:
        for s in path + [state]:
            print(s[:3])
            print(s[3:6])
            print(s[6:])
            print()
        break

    blank = state.index(0)
    row, col = divmod(blank, 3)

    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
        nr, nc = row + dr, col + dc

        if 0 <= nr < 3 and 0 <= nc < 3:
            new_blank = nr * 3 + nc
            new_state = list(state)
            new_state[blank], new_state[new_blank] = (
                new_state[new_blank], new_state[blank]
            )
            new_state = tuple(new_state)

            if new_state not in visited:
                visited.add(new_state)
                queue.append((new_state, path + [state]))
```

OUTPUT:

```
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

2. Write the Python program to solve the 8-Queen problem (CO1).

```
def solve(row, queens):
    if row == 8:
        for r in range(8):
            print("." * queens[r] + "Q" + "." * (7 - queens[r]))
        return True

    for col in range(8):
        safe = True

        for r in range(row):
            if (queens[r] == col or
                    abs(queens[r] - col) == row - r):
                safe = False
                break

        if safe:
            queens[row] = col
            if solve(row + 1, queens):
                return True

    return False

solve(0, [0] * 8)
```

OUTPUT:

```
Q.......
....Q...
.......Q
.....Q..
..Q.....
......Q.
.Q......
...Q....
```

3. Write the Python program for the Water Jug problem (CO2).

```
from collections import deque

start = (0, 0)
queue = deque([start])
visited = {start}
parent = {}

while queue:
    a, b = queue.popleft()
```

```
    if a == 2:
        path = []
        state = (a, b)

        while state in parent:
            path.append(state)
            state = parent[state]

        path.append(start)
        for state in reversed(path):
            print(state)
        break

    moves = [
        (4, b), (a, 3), (0, b), (a, 0),
        (a - min(a, 3 - b), b + min(a, 3 - b)),
        (a + min(b, 4 - a), b - min(b, 4 - a))
    ]

    for next_state in moves:
        if next_state not in visited:
            visited.add(next_state)
            parent[next_state] = (a, b)
            queue.append(next_state)
```

OUTPUT:

(0, 0)
(4, 0)
(1, 3)
(1, 0)
(0, 1)
(4, 1)
(2, 3)

4. Write the Python program for the Crypt-Arithmetic problem (CO2).

```
from itertools import permutations

letters = "SENDMORY"

for digits in permutations(range(10), 8):
    d = dict(zip(letters, digits))

    if d['S'] == 0 or d['M'] == 0:
        continue
```

```
    SEND = d['S']*1000 + d['E']*100 + d['N']*10 + d['D']
    MORE = d['M']*1000 + d['O']*100 + d['R']*10 + d['E']
    MONEY = (d['M']*10000 + d['O']*1000 + d['N']*100
             + d['E']*10 + d['Y'])

    if SEND + MORE == MONEY:
        print("SEND =", SEND)
        print("MORE =", MORE)
        print("MONEY =", MONEY)
  break
```

OUTPUT:

SEND = 9567
MORE = 1085
MONEY = 10652

5. Write the Python program for the Missionaries and Cannibals problem (CO3).

```
from collections import deque

start = (3, 3, 0)
goal = (0, 0, 1)
moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

queue = deque([(start, [])])
visited = {start}

while queue:
    (m, c, boat), path = queue.popleft()

    if (m, c, boat) == goal:
        for state in path + [(m, c, boat)]:
            print(state)
        break

    for x, y in moves:
        if boat == 0:
            nm, nc, nb = m - x, c - y, 1
        else:
            nm, nc, nb = m + x, c + y, 0

        if not (0 <= nm <= 3 and 0 <= nc <= 3):
            continue

        if ((nm > 0 and nm < nc) or
                (3 - nm > 0 and 3 - nm < 3 - nc)):
            continue
```

```
        next_state = (nm, nc, nb)

        if next_state not in visited:
            visited.add(next_state)
            queue.append((next_state, path + [(m, c, boat)]))
```

OUTPUT:

(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)

6. Write the Python program for the Vacuum Cleaner problem (CO3).

```
rooms = {'A': 'Dirty', 'B': 'Dirty'}
position = 'A'

while 'Dirty' in rooms.values():
    print("Vacuum at", position)

    if rooms[position] == 'Dirty':
        rooms[position] = 'Clean'
        print(position, "cleaned")

    elif position == 'A':
        position = 'B'
    else:
        position = 'A'

print("All rooms are clean!")
```

OUTPUT:

Vacuum at A
A cleaned
Vacuum at A
Vacuum at B

B cleaned
All rooms are clean!

7. Write the Python program to implement Breadth-First Search (BFS) (CO3).

```
from collections import deque

graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': [],
   'F': []
}

def bfs(start):
   visited = {start}
   queue = deque([start])

   while queue:
      node = queue.popleft()
      print(node, end=" ")

      for neighbour in graph[node]:
         if neighbour not in visited:
            visited.add(neighbour)
            queue.append(neighbour)

bfs('A')
```

OUTPUT:

A B C D E F

8. Write the Python program to implement Depth-First Search (DFS) (CO3).

```
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': [],
   'F': []
}

visited = set()
```

```
def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbour in graph[node]:
            dfs(neighbour)

dfs('A')
```

OUTPUT:

A B D E C F